DDoS Attack Detection and Mitigation in 5G Networks using P4 and SDN

Diana Pineda Andrade*, Kemal Akkaya†, Alexander Perez-Pons*, Selcuk Uluagac†, Abdulhadi Sahin‡
*Dept. of Electrical and Computer Engineering, Florida International University, Miami, FL USA 33174
Email: {dpine033, aperezpo}@fiu.edu
†Computer Science Department, Florida International University, Miami, FL USA 33174
Email: {kakkaya, suluagac}@fiu.edu
‡IT CIARA Department, Florida International University, Miami, FL USA 33174
Email: absahin@fiu.edu

2024 IEEE 49th Conference on Local Computer Networks (LCN) | 979-8-3503-8800-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/LCN60385.2024.10639648

***Abstract*—5G is expected to support numerous Internet of Things (IoT) devices. However, the inherent vulnerabilities and limited resources of IoT devices make them susceptible to compromise and exploitation, potentially leading to Distributed Denial of Service (DDoS) attacks on 5G infrastructure from within. While conventional Intrusion Detection Systems (IDS) can assist, 5G's unique protocols, such as the General Packet Radio Service (GPRS) Tunneling Protocol User Plane (GTP-U), pose challenges due to the inability to analyze packet headers. Therefore, we propose using Software-Defined Networking (SDN), Machine Learning (ML), and programmable switches that utilize the Programming Protocol-independent Packet Processors (P4) language to analyze GTP traffic on the fly for DDoS attack detection. Our framework enhances the efficiency of DDoS attack detection and mitigation, as demonstrated through evaluations on an actual 5G testbed using real datasets. Compared to an alternative solution that forwards GTP packets to the SDN controller, our method significantly reduces attack detection time while enhancing throughput on the SDN switch.**

***Index Terms*—5G, DDoS, P4, SDN, ONOS, GTP, IDS, IoT**

I. Introduction

Over the past decade, the proliferation of connected devices has resulted in an exponential increase in data traffic, playing a crucial role in maintaining societal interconnectedness and facilitating communication. These devices span a wide range, from IoT devices to smartphones, PCs, and more. Consequently, numerous solutions have been proposed to accommodate the growing number of devices and manage the escalating data traffic volumes [1]. One of these solutions is the deployment of 5G networks, presented as a new era of connectivity, promising a higher data rate, reduced latency, and massive device connectivity compared to its predecessors [2]. 5G networks represent an emerging era with promising capabilities for supporting Massive Machine Type Communication (mMTC), connecting a large number of devices, up to one million per square kilometer, to meet the demands of a world that is increasingly connected [3].

While 5G networks offer numerous advantages, the large-scale deployment of these networks with heterogeneous user devices raises security concerns. Attackers may exploit these benefits for malicious purposes. The extensive machine-type communications that enable the deployment of IoT devices, such as cameras, drones, and phones, can lead to security threats affecting the 5G network domains [4].

In particular, there are numerous security threats within the User Plane domain, particularly through IoT protocol-based attacks. Given the expected surge in IoT device adoption within 5G networks [5], these devices, often characterized by limited security capabilities and resources, represent significant threats to the infrastructure. Once compromised, they can become internal threat actors capable of triggering large-scale Distributed Denial of Service (DDoS) attacks [6]. These attacks may exploit traditional IP attacks such as SYN flood, UDP flood, ICMP flood, and DNS flood, using IoT protocols like MQTT and SOAP. These protocols often obscure packet headers within a GPRS Tunneling Protocol User Plane (GTP-U), employed between 5G base stations and the User Plane Function (UPF). The objective is to target the 5G network infrastructure, leading to large-scale service failures [7].

Since these attacks involve substantial traffic from compromised IoT devices, deploying an Intrusion Detection System (IDS) capable of analyzing GTP-U traffic to detect anomalies in 5G networks is crucial. One of the most popular IDS types employs Deep Packet Inspection (DPI) or stateful protocol analysis to detect attacks in network traffic [8]. This approach involves examining the content of data packets as they pass through a checkpoint on the network. However, implementing DPI becomes challenging when traffic is encrypted, particularly in high-speed IP networks where this technique can become a bottleneck. In contrast, flow-based IDS addresses the limitations of DPI by using network flow records to identify typical or malicious behavior. This method is often a more straightforward and faster choice for scenarios where high speed is essential. Several researchers have utilized Software-Defined Networking (SDN) as an IDS. SDN employs OpenFlow as a standard for packet matching functions and flow tables, allowing decision logic based on specific entries. While this approach is practical in traditional networks, it has limitations when applied to other networks, such as 5G. For instance, OpenFlow-based IDS has limited matching fields available only for specific layer headers: data-link, network, and transport. Since the 5G GTP-U layer operates above the transport layer, OpenFlow-based IDS cannot match the inner

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

IP address encapsulated within the GTP-U header. Consequently, performing packet analysis in the 5G User Plane domain to detect potential DDoS attacks becomes challenging.

Therefore, in this paper, we propose a new approach that combines SDN's benefits with programmable switches, specifically the Programming Protocol-independent Packet Processors (P4) switch. We evaluate the advantages and disadvantages of this approach in comparison to existing solutions. Our approach introduces a distributed mechanism that parses GTP packets to extract the inner IP source, destination addresses, and protocol. The goal is to deploy a flow-based IDS capable of detecting DDoS flooding attacks (such as UDP, TCP SYN, and ICMP) in 5G networks, where speed is crucial, without relying on DPI. To achieve this, the IDS analyzes network traffic characteristics—such as source and destination IP addresses, packet size, and packet frequency—to detect anomalies. We propose an integrated model that uses the P4 language to extract GTP-U packet information and enables a controller to collect this information for running a Machine Learning (ML)-based IDS. By leveraging P4's capability to extract inner IP addresses hidden within GTP packets on the fly, our approach avoids the need to forward packets to a remote controller. We describe the workflow logic implemented on the controller to make IDS decisions effectively and how the P4 switch uses this information to regularly update its rules and take mitigation actions, such as blocking a specific source IP traffic.

To evaluate our approach, we deployed a 5G-SDN-based testbed using open-source resources to simulate both the Core Network and the Radio Access Network, creating an environment that closely approximates a real-world scenario. We utilized existing datasets to train ML models, which were then implemented within the controller for real-time DDoS attack detection. We assessed performance based on the system's ability to detect attacks, the duration of detection and mitigation, and CPU resource usage. The results indicate that our technique successfully detects and mitigates flooding attacks while enhancing network performance compared to an alternative approach based on SDN and OpenFlow. Specifically, our approach significantly reduced mitigation delays and provided much greater throughput for data traffic.

Organization: The subsequent sections of the paper are organized as follows: Section II discusses related work. Section III offers background information and outlines the current problem statement addressed by our approach. Section IV details and illustrates our proposed approach. Following that, Section V presents the experimental evaluation, covering the experimental environment, performance metrics, and the baseline approach. Finally, Section VI concludes with our findings.

II. Related Work

DDoS attacks pose a significant threat to all networks. Previous research efforts have focused on IDS in traditional networks using P4 and SDN [9]–[11], but these methods are not directly applicable to 5G.

For 5G, the work in [12] introduced the 5G-NIDD dataset, developed on a 5G testbed and designed for ML and deep learning (DL) solutions for IDS. This dataset aims to detect attacks in 5G environments that could congest the network and impact infrastructure. However, it involves removing GTP headers for analysis, which strips away essential information about data traffic, making it less representative of real-world 5G network traffic. Moreover, the validation of this dataset was not tested with real-time communication flows. In contrast, our approach uses the CICDDoS2019 dataset, with a higher number of attacks, and provides a more robust evaluation.

Similar to our focus on DDoS attacks in 5G networks, the authors in [13] examined the challenges of implementing large-scale IoT environments in 5G networks and the associated risks of network failures. Their research focused on applying ML to detect DDoS attacks in 5G networks, with an emphasis on optimizing latency-related detection performance. They proposed a method for detecting DDoS attacks by classifying GTP traffic as benign or malicious. Their approach involved inputting a raw dataset into the testbed and encapsulating these packets in GTP, which reverses the typical real-world scenario. Their evaluation was focused solely on detection and included potential biases related to dataset IP addresses. In contrast, our approach operates independently of IP addresses and has been tested in real-time scenarios to counteract attacks effectively.

Finally, the authors in [14] developed an SDN-based architecture for detecting and mitigating DDoS attacks in IoT 5G Non-Standalone (NSA) networks. Their strategy involves identifying and analyzing suspicious devices using DPI within a quarantine network slice. This slice uses an extended GTP-U encapsulation and decapsulation capability via a patch to OpenvSwitch (OVS), allowing access to the data. Their solution was validated through simulations and implementing a 5G NSA testbed. In contrast, our approach was evaluated on a 5G Standalone (SA) testbed, where the P4 switch provides real-time GTP packet decapsulation for detecting and mitigating attacks by blocking the User Equipment (UE) IP address rather than quarantining the device in a network slice for further inspection and applying a patch to OVS.

III. Preliminaries and Threat Model

A. Preliminaries

1) 5G Architecture and User Plane: Given our primary focus on detecting flooding attacks within GTP traffic, it is essential to understand the workings of the 5G network's underlying components, particularly the User Plane (UP). The 5G network is based on a service-oriented architecture that includes UE (i.e., mobile devices), Next Generation Node B (gNB) (i.e., base stations), and the 5G core network. The primary role of the UP domain is to transport and deliver data between the UE and the core network, eventually reaching an external Data Network (DN). To fulfill this role, multiple connections are established among various network functions. The first connection in the UP is from the UE to the gNB, which ensures connectivity between the network and mobile devices. From the gNB, a user plane connection extends to a User Plane Function (UPF) in the core network via the N3

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

reference point. This connection may include additional hops between UPFs in the core network using the N9 reference point. Eventually, the user plane connection reaches the DN via the N6 reference point.

The UP data is transported via GTP-U over the N3 and N9 reference points [15]. GTP-U is a crucial component in 4G/5G networks, primarily used to facilitate the transmission of user data between various network elements, such as Radio Access Networks (RANs) and User Plane Functions (UPFs). GTP-U employs tunnels to carry encapsulated Tunnel Protocol Data Units (T-PDUs) and signaling messages between specific pairs of GTP-U Tunnel Endpoints. The Tunnel Endpoint ID (TEID) in the GTP header, as shown in Fig. 1, identifies the specific tunnel to which a particular T-PDU belongs.

The inner IP packet in a GTP-U packet (T-PDU) may either be an IP packet sent to a UE in the downlink direction over one or more tunnels from the external network or an IP packet sent from a UE in the uplink direction over one or more tunnels to the external network [16].

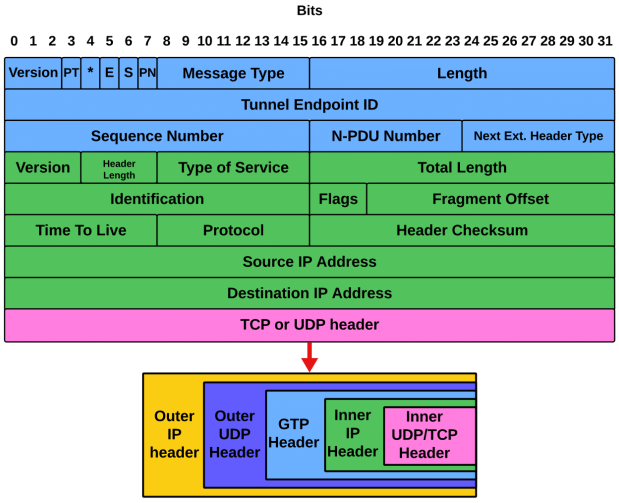


Fig. 1. GTP header

2) Software Defined Networks: The SDN paradigm is based on separating control network functions from the network devices themselves, turning the devices into simple packet-forwarding elements while the control logic is implemented in a centralized SDN controller [17]. To achieve this separation, three components are required: a centralized SDN controller, SDN-capable switches, and a management protocol such as OpenFlow or Network Configuration Protocol.

3) P4: P4 is a programming language designed for network devices such as switches and routers, enabling network operators to define and program how packets are processed and forwarded. P4 was developed to address the limitations of existing network interfaces and protocols, such as OpenFlow, which were not flexible or extensible enough to accommodate evolving network technologies and requirements.

P4 allows for specifying the header formats and field names of any protocol, as well as defining how packets are parsed, matched, and modified based on specific needs. P4 programs are implementation-independent, meaning they can be compiled and executed on various network devices, such as general-purpose CPUs, FPGAs, network processors, or ASICs. These devices are known as P4 targets, and each target must provide a compiler that maps the P4 source code into a target-specific model [18]. The P4 program consists of the following components shown in Fig. 2:

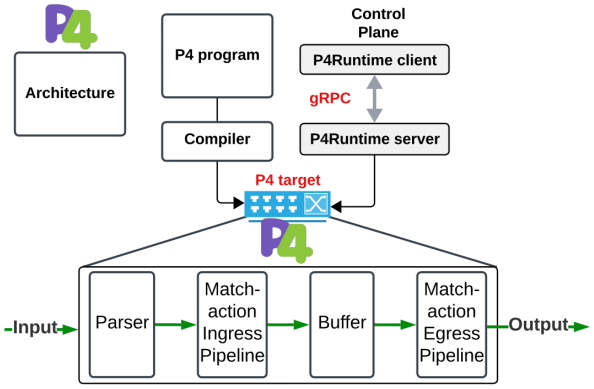


Fig. 2. P4 architecture

After defining the P4 program, it is translated into low-level instructions for the network device's data plane using a P4 compiler. This compiler takes the P4 program as input and generates a JSON file that outlines the packet-processing behavior specified by the P4 program [19]. Unlike traditional SDN, which relies on OpenFlow for communication between switches and controllers, P4 uses a gRPC server in P4Runtime for this purpose. P4Runtime is a control plane specification for managing the data plane elements of a device, allowing the P4 switch to accept P4Runtime requests [20]. This setup enables dynamic control of the data plane, providing the flexibility to adapt to different network conditions and requirements. The controller can use this communication to add flow entries to specified match tables. The only open-source controller compatible with this mechanism is the ONOS controller.

B. Threat Model

A critical security concern for 5G is the rapid proliferation of Internet of Things (IoT) devices within the 5G ecosystem. These devices, known for their limited security capabilities, can become threat actors for large-scale DDoS attacks, particularly because they are often left unattended once connected to the 5G network [7]. Once compromised, these devices can launch traditional Denial of Service (DoS) attacks, using IoT protocols to target the 5G network infrastructure. Such traditional DoS attacks include SYN floods, UDP floods, ICMP floods, DNS floods, and others [7]. These attacks target the User Plane (UP), exploiting IoT protocols to execute DDoS attacks and overwhelm the infrastructure, as illustrated in Fig. 3. Since the attacks originate from registered UEs, they are more challenging to handle than attacks from external sources.

C. Problem Description

Therefore, implementing an automated Intrusion Detection and Prevention System (IDS/IPS) for internal devices is crucial to mitigate these threats.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

Deploying an IDS for this specific 5G network use case can be challenging due to the use of GTP-U to encapsulate and transport user data. Even if a traditional IDS is deployed at the base station or using a conventional switch, analyzing the content encapsulated in GTP-U packets becomes difficult. The information within GTP-U, particularly the IP addresses from IoT UE devices, is essential for identifying the traffic source and responding to potential attacks by 5G subscribers.

The primary challenge lies in the nature of GTP-U headers, which prevents IDS solutions from effectively analyzing GTP-U packets at the base station or switch level. This limitation makes it impossible to create rules to mitigate DDoS attacks because the source IP addresses cannot be identified, even if they are registered as 5G UE devices. This contrasts with other network types where packet information can be examined in headers to identify sources. While sources might be spoofed or hidden behind proxies in other networks, this is not the case in 5G networks, where all devices must be registered in advance. This complexity underscores the need for innovative solutions to address the unique challenges posed by GTP-U headers in 5G networks.

IV. Proposed Approach

A. Motivation and Overview

Due to the challenges associated with handling GTP, one potential solution is to utilize SDN, which can serve as a viable IDS solution due to its centralized control and visibility over the network [21]. Specifically, an OpenFlow switch uses a packet-matching function and a flow table to implement decision logic based on matching specific entries and directing the matched packets to an action box. However, OpenFlow-based IDS has limited matching fields related to analyzing specific layer headers: data-link, network, and transport, as shown in Fig. 4. Since the GTP layer is above the transport layer, the OpenFlow-based IDS does not offer matching fields for the inner IP address encapsulated within the GTP header, as illustrated in Fig. 4.

Consequently, we explore alternatives within the SDN framework. One such alternative is programmable devices that use the P4 language to define the packet forwarding process. P4 allows for a detailed description and definition of the data plane behavior of network devices. While maintaining the core concept of SDN—separating the control plane from the forwarding plane—P4 also enables communication with a controller to dynamically create flow entries in tables and collect statistics for each flow. In this context, P4 inspects GTP packets, determines whether they should be forwarded, and collects statistics for each corresponding flow. Meanwhile, the controller dynamically creates flows for the GTP table. This distributed approach avoids the need for the controller to mirror every incoming packet for DPI, ensuring efficient operation. By providing a high degree of flexibility and control over packet processing and forwarding, this approach offers a robust solution for implementing an effective IDS.

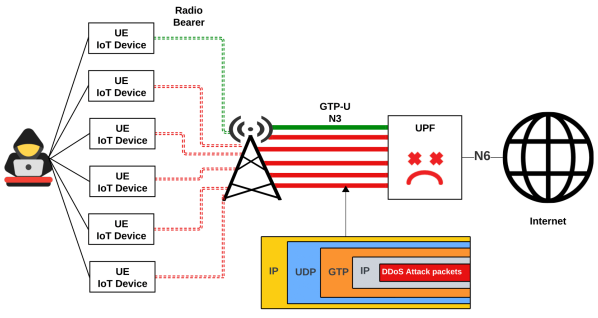


Fig. 3. Threat Model: DDoS attack encapsulated in GTP targeting the UPF.

Our proposed methodology leverages the capabilities of SDN and programmable switches to introduce a flow-based IDS for detecting and mitigating flooding attacks encapsulated within GTP-U traffic in a 5G network environment. To this end, P4-based switches (stratum) can be deployed at 5G base stations and communicate with a remote controller.

Since we are utilizing the P4 stratum switch, we selected ONOS [22] as our SDN controller since it is compatible with the P4Runtime API.

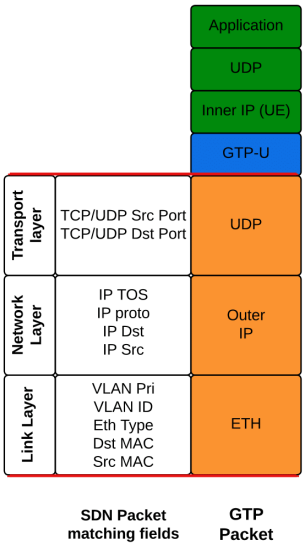


Fig. 4. OpenFlow Matching Fields for GTP protocol layers.

The detection of DDoS attacks is made possible by utilizing ML models sitting on an ML Module on top of the ONOS controller, which maintains constant communication with the ML module. As will be elaborated in the experiment section, we followed an ensemble approach that utilized two widely used ML classifiers for improved accuracy. These ML classifiers analyze incoming data from the P4 switch, which includes flow-based statistics, and provide a classification based on the input data to determine if the flow is benign or an attack. The novelty of our approach lies in the ability to pull flow statistics from the P4 switch in real time.

In the context of intrusion prevention, our approach can implement countermeasures based on feedback from the ONOS controller. For example, it enables creating and deploying rules to block specific UE IP addresses in response to identified DDoS threats. While other countermeasures could also be considered, we will focus on packet dropping as a use case in the remainder of the paper. The code and setup for the proposed approach are available on GitHub[1].

B. P4 Workflow

Our IDS approach starts at the P4 switch to handle the incoming packets by analyzing them, providing statistics to

[1]https://github.com/adwise-fiu/5G-SDN-DDoS-Detection-Mitigation

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

the controller, and creating rules. The overall algorithm flow is illustrated in Fig. 5, and has the following steps:

Packet Arrival and Parsing: When a packet is received at the switch's ingress port, it is parsed to extract the relevant header information. Note that we can sequentially parse the Ethernet, IPv4, UDP, and GTP headers until the inner IPv4 header is reached.

Match-Action Processing: The parsed headers are matched against the defined rules, if any, representing the logic of how the packets are being handled and processed within the network switch. For this purpose, the P4 program consists of five tables below:

gtp_check table: This table examines whether an incoming packet is GTP based on UDP destination and source ports.

dropped_inner_ipv4 Table: This table contains all the blocked UE IP addresses post-mitigation of DDoS flooding attacks. This table marks packets for dropping if their inner IPv4 address matches any of the entries. Otherwise, further evaluation occurs in the following tables.

gtp_flows table: If the inner IP address is not marked for dropping, the packet is evaluated in the GTP flows table based on inner IPv4 headers (source address, destination address, and protocol). If a matching flow is found, the packet proceeds to the *ipv4_check table*.

acl_table table: This table sends or clones packets to the controller. If the GTP packet does not match an existing entry in the *gtp_flow* table, automatically, this will send the packet to the controller to dynamically create a flow (forward and backward) for future communication.

ipv4_check table: The P4 switch is responsible for processing the GTP traffic and handling and forwarding the communication between the network functions. This implies that network functions in the control plane, the gNB, UE, and user plane communications are managed here. GTP packets

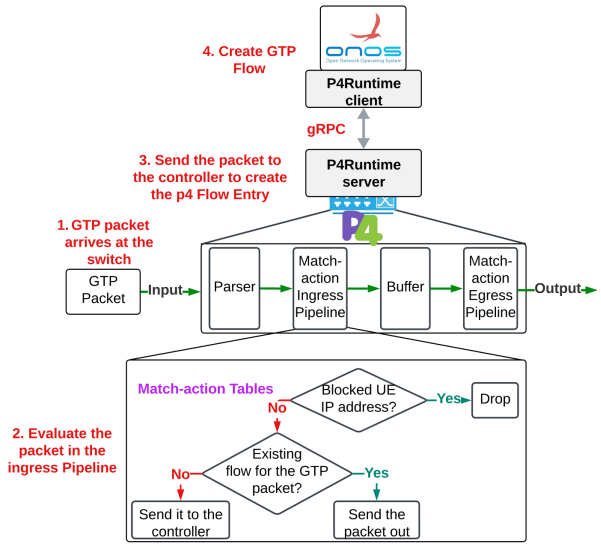


Fig. 5. P4 Workflow for analyzing the GTP packets.

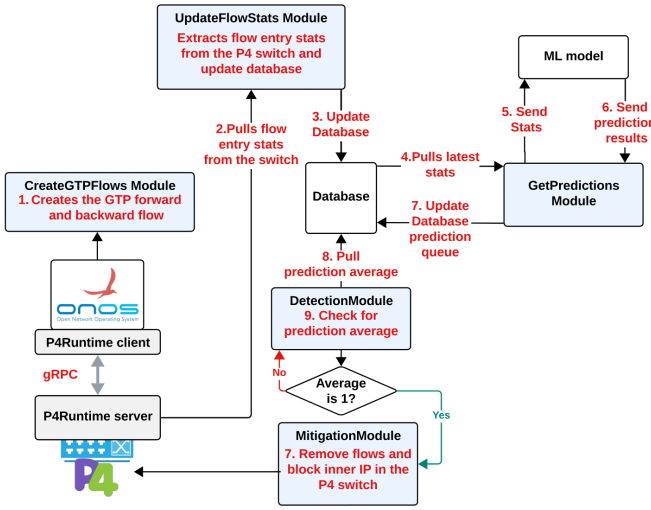


Fig. 6. ONOS Controller Workflow for DDoS Detection/Mitigation.

are only forwarded in this table if an existing flow exists in the *gtp_flows* table.

Action Execution: After the match-action process, the switch executes the specified actions, which may include forwarding packets to a specific port or dropping the packet.

Packet Egress: The packet is then forwarded to the appropriate egress port, such as the SDN controller or a specified egress port.

Note that the *gtp_flow* table allows the P4 switch allows for the counting of total packets, bytes, and duration per flow entry. These statistics can later be requested by the ONOS controller using the P4Runtime protocol. The collected data is then used to calculate the necessary input for our ML model. The P4Runtime API serves as a control plane specification for managing data plane elements in a device or program, as defined by a P4 program [23]. The P4 program implements P4Runtime and operates a gRPC server [18]. In this setup, the ONOS controller functions as the client, while the P4 switch acts as the server, allowing the controller to access details about the P4 switch.

C. Controller Workflow

The ONOS controller creates flows for the *gtp_flow* table dynamically based on the traffic coming from the UE (forward and backward). The workflow, illustrated in Fig. 6, uses the capabilities and flexibility of ONOS to design various modules for a specific application. The approach incorporates five main modules: *CreateGTPFlows, UpdateFlowStats, GetPredictions, DetectionModule,* and *MitigationModule*. Together, these modules facilitate GTP flow management, P4 statistics gathering, additional feature calculation for flow-based predictions, transmission of flow statistics to ML models, periodic tracking of predictions, and mitigation of attacks by creating flow entries to block specific UE IP addresses on the P4 switch. Below, we describe the details of the workflow and the functions of each module mentioned above:

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

GTP Packet Processing at ONOS controller: Once the GTP packet arrives at the ONOS controller, it undergoes parsing to extract the inner IPv4 headers (source IP address, destination IP address, and protocol). Using this extracted information, the ONOS controller creates forward (source-destination-protocol) and backward (destination-source-protocol) flows as FlowRules and adds them to the P4 switch. These flows are also recorded in a database. This process is managed by the *CreateGTPFlows* module, which is responsible for extracting the necessary information, creating flows in the P4 switch and database, and maintaining a list of FlowRules for potential removal during the mitigation process.

Extracting P4 statistics and update: Concurrently with packet processing, the ONOS controller periodically extracts flow entry statistics from the P4 switch in the *gtp_flow* table. The extracted metrics include total packets, duration, and bytes associated with each flow. The controller then sends this data to the database, where additional metrics such as FlowBytes per second are calculated. This process ensures that the flow information dynamically reflects real-time network behavior. This task is managed by the *UpdateFlowStats* module.

Obtaining predictions based on the updated statistics: The *GetPredictions* module retrieves updated flow statistical data from the database periodically. It extracts the features required for predictions from the ML module. Once the module receives the ML predictions, it updates the database, which uses a queuing mechanism to store the latest k predictions and compute their average. This averaged value is a key indicator for the mitigation module.

Detection Process: The *DetectionModule* has the task of periodically retrieving the average of the prediction queues of the ML modules. The mitigation module raises a flag if they all agree on an attack. This adds the source IP (UE IP) to a list that the *MitigationModule* will use.

Mitigation Process: The *MitigationModule* constantly checks the flagged IP addresses to block them. It removes all the existing flows in the *gtp_table* and the database, and adds the flagged UE IPs to the *dropped_ inner_ipv4* table and the database. This action blocks the specific UEs and prevents the addition of new flows to the *gtp_flow* table.

V. Experimental Evaluation

A. Experiment Environment

We developed a 5G-SDN-based testbed for our experimental setup using open-source projects: Open5GS for the Core Network and UERANSIM for the 5G UE and Radio Access Network (gNB). Our setup includes two VMs running Ubuntu 20.04 with virt-manager, which are used to deploy four additional VMs representing different components of the 5G network. These components are one VM for the 5G Core Network Functions, another for the 5G User Plane Function (UPF), a third for simulating the gNB, and a fourth for simulating multiple UEs. To ensure that our testing environments do not interfere with one another, we segregated the network infrastructure across the two VMs. The first VM hosts the P4 stratum switch, which communicates with the ONOS controller, as illustrated in Fig. 7, while the OVS communicates with the RYU controller.

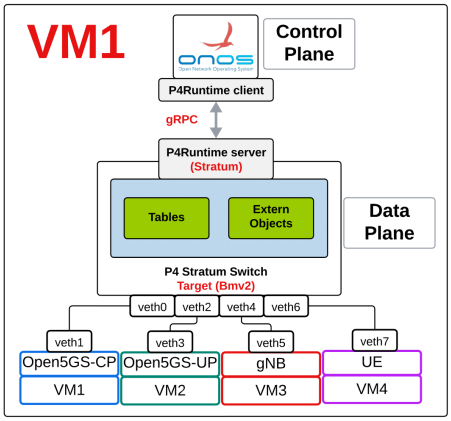


Fig. 7. Proposed Approach Testing Environment

B. Performance Metrics

We compare key metrics, including CPU consumption, attack detection, and mitigation time. Given the high demands of 5G networks, evaluating these metrics is crucial to ensure both approaches can handle the required bandwidth and operational efficiency. Additionally, we focus on the latency between threat detection and mitigation, as this is critical for maintaining the security of the network infrastructure.

CPU utilization: This metric measures the amount of processing power consumed by the switch and SDN controller. This metric allows us to assess the efficiency of the network processing and resource utilization.

Detection Time: It is the duration it takes to have a DDoS attack prediction on average. This measures how quickly the system can recognize and flag potential DDoS flooding attacks.

Mitigation time: This is the time after detecting a DDoS attack, removing all the flows, and blocking the UE IPs to respond to the threat. This metric is important for assessing the system's speed in implementing countermeasures.

Regarding the ML performance evaluation, the metrics considered are accuracy, precision, recall, and F1-score. Considering all these metrics, we can evaluate our selected ML algorithms in detecting DDoS attacks in 5G networks.

C. Baseline approach: OVS-RYU

To assess the effectiveness of our novel methodology, it is crucial to compare it with a baseline based on OpenFlow standards. We selected the OVS-RYU-based SDN approach as detailed in [24], with enhancements to enable tracking GTP flows and detecting and mitigating DDoS attacks. This method directs all GTP packets to the controller for thorough examination due to OpenFlow's limitations in accommodating matching fields related to headers encapsulated within GTP traffic. Consequently, the RYU controller is responsible for decapsulating the GTP header to extract the UE IP address.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DDoS DETECTION AND MITIGATION TIME RESULTS FOR BOTH APPROACHES ON 5G.

Case	Period	ONOS-P4 (s)		ONOS	P4	RYU-OVS (s)		RYU	OVS
		Detection	Mitigation	CPU % avg	CPU % avg	Detection	Mitigation	CPU % avg	CPU % avg
ICMP	2s	39	2	5.45%	58.55%	103	2	51.23%	6.32%
UDP		20	2	5.02%	28.36%	81	2	21.16%	3.33%
SYN		22	4	5.19%	28.76%	87	2	18.58%	2.99%
ICMP	5s	49	4	3.88%	55.92%	255	2	57.16%	5.35%
UDP		33	4	3.89%	24.40%	228	2	22.72%	3.66%
SYN		33	4	4.19%	29.00%	251	2	22.40%	3.48%
ICMP	10s	85	3	3.36%	60.18%	473	2	37.77%	3.70%
UDP		62	3	3.55%	29.69%	444	2	15.77%	2.62%
SYN		49	3	2.97%	29.23%	499	2	20.97%	3.30%
ICMP	20s	140	2	2.24%	48.57%	965	2	38.47%	3.71%
UDP		98	3	2.31%	30.22%	957	2	21.60%	3.45%
SYN		94	3	2.09%	30.05%	957	2	20.46%	3.26%

D. Experiment Results

1) ML Model Performance: Integrating ML into the SDN environment involves several key steps. This subsection will focus on building the ML model, which will be used in the SDN approaches as a classifier to make predictions based on real-time flow data (i.e., flow-based IDS). Based on various past works related to ML models, we have identified several critical steps to consider when building an ML model, which are covered below:

a) Data Collection/Selection: The objective of our approach is to detect DDoS flooding attacks based on statistical flow data extracted from the SDN environment (e.g., duration, total packets, and bytes). For our approach, we used available datasets with extensive data to predict a wider range of attacks and prevent model bias. Several benchmark datasets for training and testing IDS are available. Two commonly used ones are the DARPA (1998-1999) and KDD'99 Cup datasets, which contain outdated attacks, potentially impacting their effectiveness in current approaches. Others, such as the NSL-KDD dataset, still have some drawbacks of the KDD Cup, while the CAIDA (2002-2016) dataset lacks common attacks and effectiveness in identifying individual attacks. Another frequently used dataset in IDS is the UNSW-NB-15 dataset, which provides nine attack classes. However, we sought a dataset with more attacks and flow-based features, leading us to choose the CICDDoS2019 dataset, which offers various datasets for different cybersecurity solutions, with the most modern DDoS attacks [25]. This dataset contains benign data and the most common DDoS attacks, collected over two different days, with twelve DDoS attacks on the training day and seven attacks on the testing day [26].

b) Feature Selection and Data Processing: Given that our available data is intrinsically flow-based, we chose twenty features emphasizing flow-based characteristics over packet-based ones from the original set of 88 features in the CICDDoS2019 dataset. These selected features, computable from total packets, bytes, and duration data, were chosen to best capture the essence of flow behavior. Since many of the twenty flow-based features exhibit correlations, we conducted a correlation matrix to identify highly correlated features. Subsequently, we dropped these features to prevent overfitting and refined our feature set to ten non-correlated features: Protocol, Flow Duration, Total Forward and Backward Packets, Total Length of Forward and Backward Packets, Flow Bytes per second, Backward Packets per second, Minimum and Maximum Backward Packet Length.

Once the features were selected, we proceeded to the data processing phase. Our training data consists of the first-day dataset, containing instances of twelve different attacks, while the testing data is derived from the second day and involves seven types of attacks. After dropping unnecessary columns, we labeled and encoded the data to facilitate binary classification, distinguishing between benign and attack flows. Subsequently, we cleaned the data by addressing missing values, replacing infinite values, and eliminating duplicate rows to ensure the integrity and quality of the dataset.

c) Model Selection and Training: Given our model's binary classification nature, we evaluated four well-known classifiers: Random Forest (RF), Naïve Bayes (NB), Logistic Regression (LR), and K-Nearest Neighbors (KNN) [25]. Following the data cleaning process, we used the scikit-learn library to fit the model.

d) ML Model Evaluation: After training the model, the performance evaluation of each classification algorithm will be determined using the mentioned metrics. It is important to note that the obtained results for the model evaluation are based on the training and testing phases, also known as offline evaluation. The results are summarized in Table II:

TABLE II
ML MODEL EVALUATION FOR DDoS ATTACKS

	Accuracy	Precision	Recall	F1-Score
RF	40.84%	99.56%	37.88%	54.88%
NB	98.04%	98.15%	99.82%	98.98%
LR	97.26%	97.30%	99.88%	98.57%
KNN	50.44%	48.28%	48.28%	64.92%

LR exhibits strong performance with an accuracy of 97.26%, precision of 97.30%, recall of 99.88%, and an F1-score of 98.57%. In contrast, KNN achieves a lower accuracy of 50.44% despite high precision (99.05%). However, its low recall (48.28%) yields a modest F1-score of 64.92%. On the other hand, NB demonstrates solid performance, with an accuracy of 98.04%, precision of 98.15%, recall of 99.82%, and an F1-score of 98.98%. Meanwhile, RF records an accuracy of 40.84%, with high precision (99.56%) but low

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

recall (37.88%), resulting in an F1-score of 54.88%. The poor performance of RF and KNN with the selected flow-based features may be attributed to their inability to capture the complex relationships within the data effectively. Flow-based features are not discriminative enough to distinguish between benign and attack behaviors, reflected in algorithms such as RF, KNN, and others [27].

Based on the results, Logistic Regression and Naïve Bayes outperformed Random Forest and KNN. Given these findings, we will utilize Logistic Regression and Naïve Bayes to predict flooding attacks in our tests on the 5G testbed.

2) P4 Performance Results: We tested our approach against various DDoS flooding attacks, specifically ICMP, UDP, and TCP SYN. These attacks were simulated under controlled conditions to measure detection and mitigation times. For the ICMP flooding attack, we used the ping utility with sudo privileges to select a small interval that simulates the flooding and bind the flooding to the UE interface (uesimtun0). In the context of UDP and TCP SYN, we developed custom C++ programs to simulate these two DDoS flooding attacks, with a small interval between the sent packets.

The experiment focused on measuring detection and mitigation times under DDoS attack scenarios, specifically exploring ICMP, UDP, and TCP SYN attacks. It is crucial to note that we initiate the attack in our approach, thereby generating a flow. The detection time essentially signifies when the attack commenced and the duration it took to detect it. To reduce false positives, our detection process involves five consecutive predictions of 1 (classified as an attack) from both ML algorithms. The objective is to evaluate how the prediction frequency intervals impact attack detection and mitigation times. To achieve this, we varied the frequency intervals for requesting P4 data and obtaining ML predictions to identify attacks every 2, 5, 10, and 20s. Additionally, CPU utilization for both approaches is monitored during this experiment.

The detection and mitigation time assessment uses a Python script, which continuously checks for removed flows from the database and newly added blocked IP addresses. The script extracts the removed flow duration to determine the detection time and tracks the time to block each IP address, representing the mitigation time. The recorded metrics include detection time, mitigation time, and average CPU utilization for different components in both ONOS-P4 and OVS-RYU setups.

Table I results show that our proposed ONOS-P4 approach consistently outperformed the RYU-OVS setup in terms of detection time. The ONOS-P4 strategy exhibited outstanding efficiency, with minimal detection times of 39s for UDP flood, 20s for SYN flood, and 22s for ICMP flooding attacks. In contrast, the OVS-RYU approach lagged significantly, registering comparatively prolonged detection periods of 81s for UDP, 87s for SYN, and 103s for ICMP attacks for cases where the predictions are made every 2s. The mitigation times results remained relatively low across various attack frequencies and intervals, reflecting efficient response mechanisms.

This discrepancy can be attributed to several factors since there was a big difference between these two approaches and the detection performance. The OVS-RYU relies on a centralized decision-making process in the controller with fixed polling intervals and an external database for flow table management. This introduces latency and overhead to the approach as the number of flows increases. Since the RYU controller relies on external dependencies, such as synchronous communication with the database, the ML model, detection, and mitigation module, these can further increase the detection times. In contrast, the ONOS-P4 approach employs a distributed decision-making process and programmable data plane, enabling efficient control over flow rules within P4-capable switches. The distributed architecture, asynchronous communication with the ML model, and integration of cleanup tasks into scheduled processes contribute to faster detection and mitigation in the ONOS-P4 framework.

Additionally, it is important to note the comparable CPU utilization results of this experiment between the P4 switch and the RYU controller, both responsible for packet decapsulation. This similarity suggests that, despite their distinct roles, the CPU demands for these components align closely. Furthermore, the OVS-RYU approach forwards each packet to the controller, significantly reducing the throughput on the UPF switch due to the re-routing introduced. On the other hand, our approach processes the packets on the fly without re-routing the flows, maintaining higher throughput.

In summary, this comprehensive evaluation positions the ONOS-P4 combination as a promising choice for SDN security. It offers persistent throughput and efficient attack detection and mitigation in the context of flooding and 5G user data traffic encapsulated within the GTP tunnel.

VI. Conclusion

In this paper, we introduced a novel approach that utilizes programmable switches to analyze GTP traffic to detect and mitigate DDoS attacks from compromised IoT devices within 5G networks. Our proposed approach integrates a P4 switch and ONOS controller within a model that processes GTP packets on the fly. We developed workflow logic to leverage ML-based classifiers without requiring each GTP packet to be sent through the P4 switch. Evaluations performed in a 5G testbed demonstrated significant advantages over the baseline OVS-RYU solution. Notably, our approach reduced detection time to 20 seconds, compared to 81 seconds for the OVS-RYU method. These results highlight the benefits of the distributed ONOS-P4 architecture over the centralized OVS-RYU approach.

Acknowledgment

This research was funded by a National Centers of Academic Excellence in Cybersecurity grant (H98230-21-1-0324 (NCAE-C-002-2021)), which is part of the US National Security Agency and National Science Foundation under the grant #2147196.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] H. Tang, N. Yang, Z. Zhang, Z. Du, and J. Shen, *5G NR and Enhancements: From R15 to R16*. Elsevier, 2021.

[2] D. Chandramouli, R. Liebhart, and J. Pirskanen, *5G for the Connected World*. John Wiley & Sons, 2019.

[3] S. Sicari, A. Rizzardi, and A. Coen-Porisini, "5g in the internet of things era: An overview on security and privacy challenges," *Computer Networks*, vol. 179, p. 107345, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128620300827

[4] I. Ahmad, T. Kumar, M. Liyanage, J. Okwuibe, M. Ylianttila, and A. Gurtov, "Overview of 5g security challenges and solutions," *IEEE Communications Standards Magazine*, vol. 2, no. 1, pp. 36–43, 2018.

[5] I. Ahmad, S. Shahabuddin, T. Kumar, J. Okwuibe, A. Gurtov, and M. Ylianttila, "Security for 5g and beyond," *IEEE Communications Surveys Tutorials*, vol. 21, no. 4, pp. 3682–3722, 2019.

[6] S. Sicari, A. Rizzardi, and A. Coen-Porisini, "5g in the internet of things era: An overview on security and privacy challenges," *Computer Networks*, vol. 179, p. 107345, 2020.

[7] H. Kim, "5g core network security issues and attack classification from network protocol perspective." *J. Internet Serv. Inf. Secur.*, vol. 10, no. 2, pp. 1–15, 2020.

[8] M. F. Umer, M. Sher, and Y. Bi, "Flow-based intrusion detection: Techniques and challenges," *Computers Security*, vol. 70, pp. 238–254, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167404817301165

[9] N. Aslam, S. Srivastava, and M. Gore, "Onos flood defender: An intelligent approach to mitigate ddos attack in sdn," *Transactions on Emerging Telecommunications Technologies*, vol. 33, 05 2022.

[10] F. Musumeci, V. Ionata, F. Paolucci, F. Cugini, and M. Tornatore, "Machine-learning-assisted ddos attack detection with p4 language," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 2020, pp. 1–6.

[11] G. Altangerel, M. Tejfel, and E. Tsogbaatar, "Iot anomaly detection with 1d cnn using p4 capabilities," *Acta Electrotechnica et Informatica*, vol. 23, no. 2, pp. 3–12, 2023.

[12] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5g-nidd: A comprehensive network intrusion detection dataset generated over 5g wireless network," 2022.

[13] Y. Kim, Y. Kim, and H. Kim, "A comparison experiment of binary classification for detecting the gtp encapsulated iot ddos traffics in 5g network," *Journal of Internet Technology*, vol. 23, no. 5, pp. 1049–1060, 2022.

[14] D. Candal-Ventureira, P. Fondo-Ferreiro, F. Gil-Castiñeira, and F. J. González-Castaño, "Quarantining malicious iot devices in intelligent sliced mobile networks," *Sensors*, vol. 20, no. 18, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/18/5054

[15] S. Rommer, P. Hedman, M. Olsson, L. Frid, S. Sultana, and C. Mulligan, *5G Core Networks: Powering Digitalization*. Academic Press, 2019.

[16] ETSI, "3gpp ts 29.281 v8.0.0 (2008-12); 3rd generation partnership project; technical specification group core network and terminals; general packet radio system (gprs) tunnelling protocol user plane (gtpv1-u) (release 8)," 2008. [Online]. Available: https://www.etsi.org/deliver/etsi_ts

[17] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114–119, Feb. 2013, conference Name: IEEE Communications Magazine.

[18] T. P. L. Consortium, "P4 16 language specification," 2017. [Online]. Available: https://p4.org/p4-spec/docs/P4-16-v1.0.0-spec.html#sec-scope

[19] p4lang, "p4c - p4_16 compiler," 2 2024. [Online]. Available: https://github.com/p4lang/p4c

[20] T. P. A. W. Group, "P4runtime specification," 07 2021. [Online]. Available: 2

[21] G. A. Ajaeiya, N. Adalian, I. H. Elhajj, A. Kayssi, and A. Chehab, "Flow-based intrusion detection system for sdn," in *2017 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2017, pp. 787–793.

[22] ONF, "Open Network Operating System (ONOS) SDN Controller for SDN/NFV Solutions — opennetworking.org," https://opennetworking.org/onos/.

[23] p4lang, "p4lang/p4runtime: Specification documents for the p4runtime control-plane api," [1](https://github.com/p4lang/p4runtime), 2024.

[24] D. Pineda, R. Harrilal-Parchment, K. Akkaya, and A. Perez-Pons, "Sdn-based gtp-u traffic analysis for 5g networks," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, 2023, pp. 1–4.

[25] P. Singh Samom and A. Taggu, "Distributed denial of service (ddos) attacks detection: A machine learning approach," in *Applied Soft Computing and Communication Networks*, S. M. Thampi, J. Lloret Mauri, X. Fernando, R. Boppana, S. Geetha, and A. Sikora, Eds. Singapore: Springer Singapore, 2021, pp. 75–87.

[26] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*, 2019, pp. 1–8.

[27] M. A. Salahuddin, M. Faizul Bari, H. A. Alameddine, V. Pourahmadi, and R. Boutaba, "Time-based anomaly detection using autoencoder," in *2020 16th International Conference on Network and Service Management (CNSM)*, 2020, pp. 1–9.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:18 UTC from IEEE Xplore. Restrictions apply.